Reviewer Pack — Minimal Rank of Symplectic Geometry over XOR-AND Tree Width

Entry e309c0bda6bc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 20:29:37 UTC

Minimal Rank of Symplectic Geometry over XOR-AND Tree Width

Entry ID: `e309c0bda6bc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 20:29:37 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

> ['For any given XOR-AND tree, the minimal rank of its associated symplectic form is upper-bounded by a function of its width, i.e., E[rank(SymplecticForm(T)) ≤ Θ(width(T))] for all XOR-AND trees T.', 'Moreover, there exists an absolute constant c such that for every XOR-AND tree T with n ≤ 40 variables, the minimal rank of its symplectic form is at least c·width(T).', 'This holds true regardless of the distribution of clauses within the tree.']

Rationale (proposer's reasoning):

> ['Symplectic geometry has not been extensively applied to complexity theory, particularly in the context of XOR-AND trees.', 'The conjecture draws an analogy between symplectic forms and the structure of XOR-AND trees, suggesting that the geometric properties of the former could shed light on the algorithmic complexities of the latter.', 'Exploring this relationship may reveal new insights into the complexity of computational problems, such as satisfiability.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `62ded563ddd6219d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of a symplectic form associated with an XOR-AND tree is considered supported if the empirical mean rank across at least 30 random seeds is within 3 units of the expected value based on the conjectured function of width(T).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def generate_xor_and_tree(n: int, max_depth: int) -> list:
    if n <= 0 or max_depth <= 0:
        return []
    if n == 1:
        return [random.choice([0, 1])]
    if max_depth == 1:
        return [random.choice([0, 1]) for _ in range(n)]

    left_size = random.randint(1, n-1)
    right_size = n - left_size

    left = generate_xor_and_tree(left_size, max_depth-1)
    right = generate_xor_and_tree(right_size, max_depth-1)

    return [random.choice([0, 1]) for _ in range(n)]

def compute_symplectic_form(tree: list) -> list:
    n = len(tree)
    symplectic_form = [[Fraction(0, 1)] * n for _ in range(n)]

    def assign(i, j, value):
        if i < j:
            symplectic_form[i][j] = value
            symplectic_form[j][i] = -value

    def xor_and(a, b):
        return a ^ b

    def and_or(a, b):
        return a & b
```

```
    for i in range(n):
        if tree[i] == 0:
            assign(i, (i + 1) % n, Fraction(1, 1))
        else:
            assign(i, (i - 1) % n, Fraction(-1, 1))

    return symplectic_form

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    width_values = [5, 10, 15, 20, 30, 40]
    total_ranks = []
    counterexample = ""

    for width in width_values:
        tree = generate_xor_and_tree(n, width)
        symplectic_form = compute_symplectic_form(tree)

        # Compute minimal rank
        min_rank = len([row for row in symplectic_form if any(row)])
        total_ranks.append(min_rank)

    mean_value = sum(total_ranks) / len(total_ranks)
    expected_value = sum(width_values) * 0.5  # Simplified example function

    if abs(mean_value - expected_value) <= 3:
        conjecture_holds = True
    else:
        conjecture_holds = False
        counterexample = "mean_value does not match expected_value"

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean_value,
        "instances_tested": len(width_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 53))  # Default to f

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
```

```
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
es not match expected_value'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 28.5, 'instances_tested': 6, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 31.0, 'instances_tested': 6, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 31.333333333333332, 'instances_tested': 6, 'c
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 32.666666666666664, 'instances_tested': 6, 'c
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 31.0, 'instances_tested': 6, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 30.666666666666668, 'instances_tested': 6, 'c
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 30.166666666666668, 'instances_tested': 6, 'c
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 29.166666666666668, 'instances_tested': 6, 'c

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ecf6c0bd.py", line 107, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ecf6c0bd.py", line 107, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The empirical mean rank of the symplectic forms associated with XOR-AND trees does not match the expected value based on the conjectured function of w | next: Investigate the counterexamples to understand why the minimal ranks are higher than expected and refine the conjecture accordingly.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11410
2	preregistration	ollama_remote	glm4:latest	0	0	5711
3	novelty	ollama_remote	glm4:latest	0	0	4906

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	5235
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12593
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10937
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10117
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10571
9	judge	ollama_remote	glm4:latest	0	0	23880

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 95361 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e309c0bda6bc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e309c0bda6bc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e309c0bda6bc.tar.gz` (if generated)